Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

Лабораторная работа №3

по курсу **«Технологии разработки программного обеспечения»**

на тему

«Автоматическая генерация исходного кода программы и проектной документации»

Выполнил: Солдаткин Н. С. _____ (подпись) _____ (дата)

Группа: ИУ5-11М

Проверила: Виноградова М. В. _____ (подпись) _____ (дата)

Москва, 2026 г.

1 Задание

1.1 Введение

Автоматическое документирование позволяет получать справочные материалы непосредственно из исходного кода и комментариев разработчика. В лабораторной работе этот подход рассмотрен на примере Doxygen и серверного проекта Beatstore Backend на языке Python. Основная часть посвящена подготовке комментариев, настройке генератора и анализу полученной документации. В дополнительном задании рассмотрена генерация кода с помощью GitHub Copilot.

1.2 Цель работы

Изучить Doxygen и получить навыки автоматического документирования проекта, файлов, классов, методов и атрибутов. Освоить применение GitHub Copilot для реализации методов по комментариям и проверки предложенного кода.

1.3 Средства выполнения

- редактор Visual Studio Code с расширением GitHub Copilot;
- система Doxygen и браузер для просмотра HTML-документации;
- исходный код проекта Beatstore Backend на языке Python;
- библиотеки SQLAlchemy и компоненты проекта для работы с тегами и моделями музыкального каталога.

1.4 Задачи

1. Подготовить комментарии в формате Doxygen для исходного кода и сохранить файлы проекта.
2. Рассмотреть настройку проекта в Doxygen, генерацию HTML и RTF и просмотр результатов документирования.

3. Продемонстрировать краткое и подробное описание класса и его методов, входные параметры и возвращаемые значения, атрибуты класса, описание проекта и файла со ссылкой на него, а также производный класс.

4. Подготовить в Visual Studio Code проект и класс с методами и свойствами для демонстрации GitHub Copilot.

5. Продемонстрировать предложения кода по четырём видам запросов: краткому комментарию и имени функции, подробному описанию её работы, описанию алгоритма и описанию вызова функции.

6. Проанализировать предложенные реализации и определить необходимые корректировки.

2 Ход работы

Основная часть посвящена системе Doxygen. Для документирования использован проект Beatstore Backend: серверное приложение музыкального каталога на Python. Примеры описаний рассмотрены на классе `TagRepository`, а наследование – на модели `Product`.

2.1 Комментирование исходного кода по правилам Doxygen

В файле `repository.py` подготовлены специальные комментарии. Блок документации начинается с `##`, последующие строки — с `#`. Использованы команды `@file` для файла, `@brief` для краткого описания, `@details` для подробного описания, `@param` для параметров, `@return` для результата, `@var` для атрибута и `@note` для особенностей реализации [2, 3].

Фрагмент комментария к классу имеет следующий вид:

```
## @brief Репозиторий тегов.
# @details Выполняет запросы к тегам через переданную асинхронную
# сессию SQLAlchemy.
class TagRepository:
```

Аналогично описаны конструктор, методы получения тегов и атрибут `session`. Для поиска по идентификатору указаны входной параметр `tag_id`, результат `Tag | None` и загрузка связанных продуктов. Эта загрузка выполняется через `selectinload(Tag.products)`. Сохранённый файл содержит одновременно реализацию и сведения для генератора справки.

2.2 Запуск Doxygen и настройка проекта

В соответствии с методическими указаниями настройка включает название проекта, каталог исходников, рекурсивное сканирование и каталог результата. Для рассматриваемого Python-проекта выбираются файлы `.py`, а кодировка входных файлов должна соответствовать UTF-8. Русские заголовки полученных страниц отражают выбор русского языка документации.

Ниже приведён пример параметров `Doxyfile` для воспроизведения такой документации с обоими форматами вывода [4]. Пути заданы относительно корня документируемого проекта:

```
PROJECT_NAME       = "Beatstore Backend"
INPUT              = .
FILE_PATTERNS      = *.py *.dox *.md
RECURSIVE          = YES
INPUT_ENCODING     = UTF-8
OUTPUT_LANGUAGE    = Russian
OUTPUT_DIRECTORY   = docs
OPTIMIZE_OUTPUT_JAVA = YES
SOURCE_BROWSER     = YES
GENERATE_HTML      = YES
GENERATE_RTF       = YES
GENERATE_LATEX     = NO
CLASS_GRAPH        = YES
```

Параметр `SOURCE_BROWSER` обеспечивает переход к исходному тексту, а `CLASS_GRAPH` включает отображение наследования.

2.3 Генерация документации и просмотр результата

Для создания шаблона настроек и запуска генератора из корня проекта используются команды:

```
doxygen -g Doxyfile
doxygen Doxyfile
```

После настройки в графическом интерфейсе аналогичный запуск выполняется на вкладке Run кнопкой Run Doxygen. Для приведённых выше путей HTML-справка открывается через `docs/html/index.html`, а RTF-документ — через `docs/rtf/refman.rtf` в текстовом редакторе, поддерживающем RTF. Формат RTF включается параметром `GENERATE_RTF = YES` [4].

Предоставленные результаты демонстрируют просмотр HTML-страниц в браузере.

2.4 Краткое и подробное описание класса

На странице `repository.TagRepository` краткое описание «Репозиторий тегов» отображается под именем класса. Раздел «Подробное описание» уточняет, что запросы выполняются через переданную асинхронную сессию

SQLAlchemy. На рисунке 1 также перечислены конструктор, три метода чтения тегов и открытый атрибут `session`.

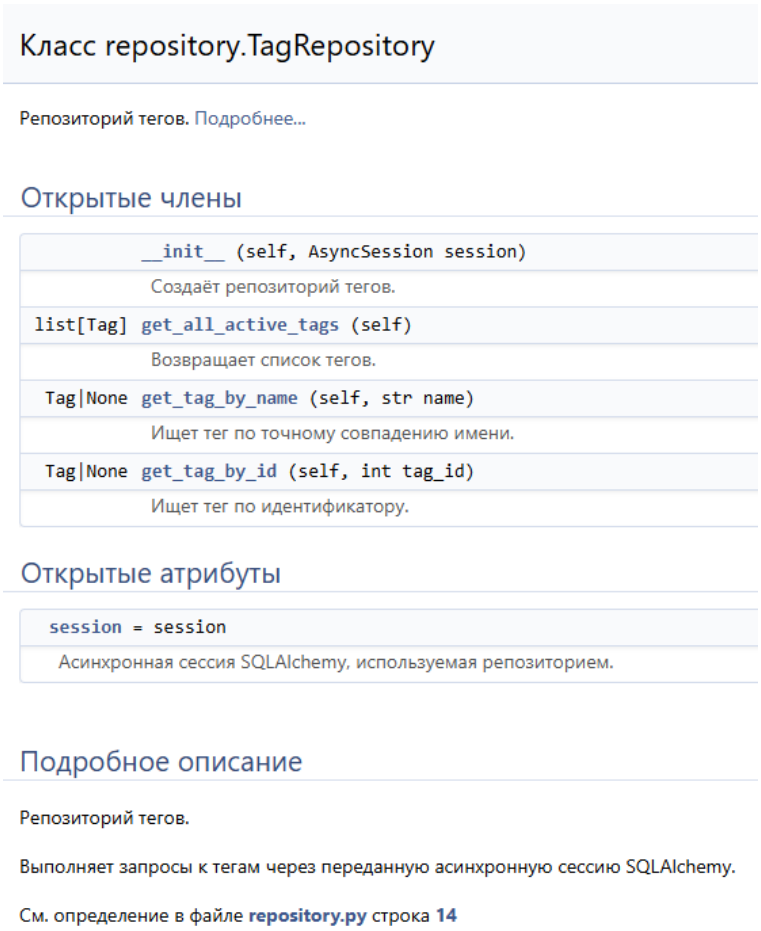


Рисунок 1 – Краткое и подробное описание класса TagRepository

Разделение описания на краткую и подробную части позволяет сначала определить назначение класса, затем ознакомиться с его зависимостями и доступными операциями.

2.5 Краткое и подробное описание функции класса

Для метода `get_all_active_tags()` краткое описание сформулировано как «Возвращает список тегов». Подробная часть поясняет фактическую реализацию – эти сведения и описание возвращаемого значения показаны на рисунке 2.

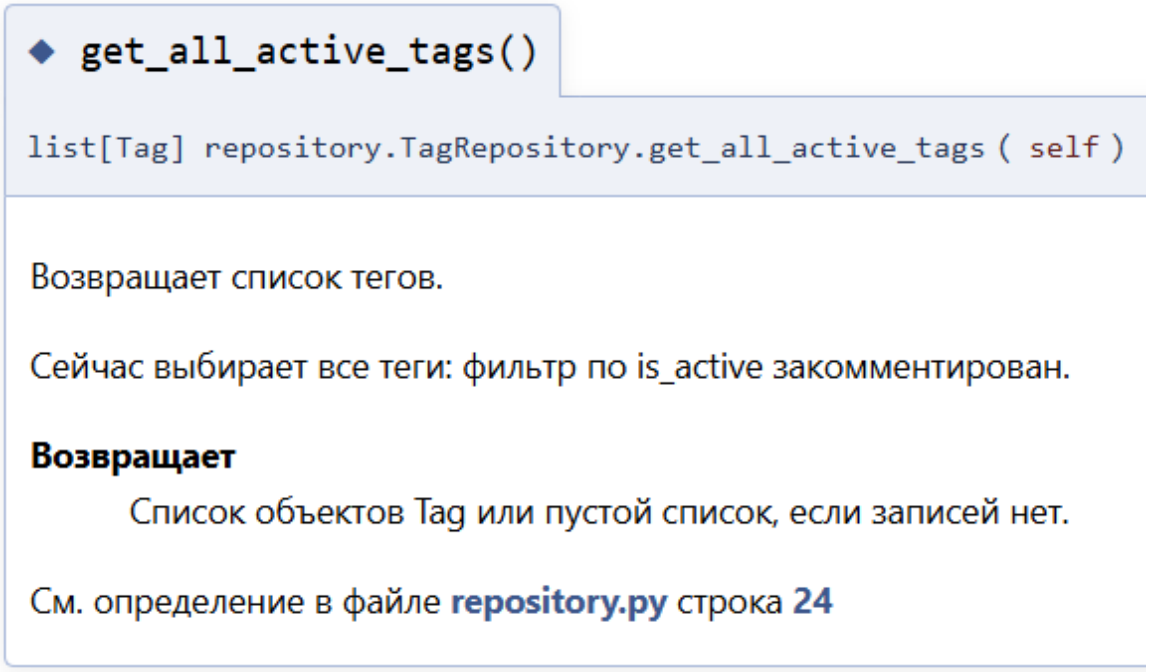


Рисунок 2 – Описание метода get_all_active_tags() и его результата

Возвращаемое значение – список объектов `Tag` либо пустой список, если записи отсутствуют.

2.6 Входные параметры и возвращаемые значения функций

На странице `get_tag_by_name()` команда `@param` формирует раздел «Аргументы»: параметр `name` описан как имя тега для поиска. Команда `@return` формирует раздел «Возвращает»: первый найденный объект `Tag` либо `None`, если совпадений нет. Результат приведён на рисунке 3.

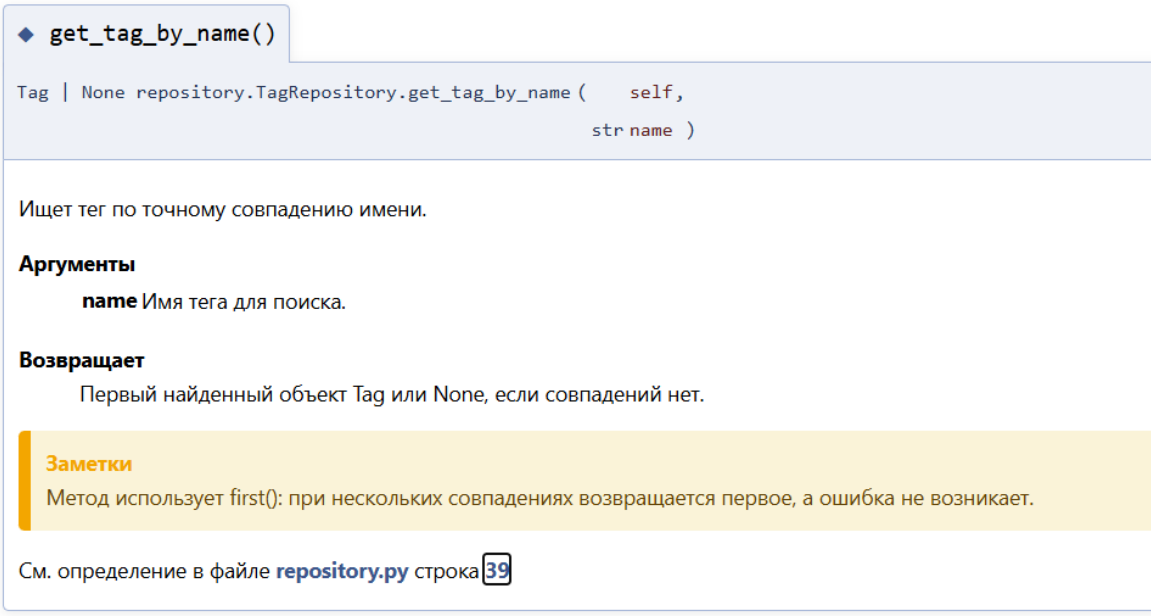


Рисунок 3 – Входной параметр, результат и заметка метода get_tag_by_name()

Заметка `@note` отдельно поясняет поведение `first()`: при нескольких совпадениях возвращается первое, исключение не возникает. Для конструктора аналогично описан параметр `session`, а для `get_tag_by_id()` – параметр `tag_id` и результат `Tag | None`.

2.7 Описание переменных класса

Атрибут `session` содержит асинхронную сессию SQLAlchemy, через которую репозиторий выполняет запросы. В исходном файле его назначение задано отдельным блоком:

```
## @var session
# @brief Асинхронная сессия SQLAlchemy, используемая репозиторием.
```

На рисунке 4 показано описание атрибута, включённое в документацию класса.

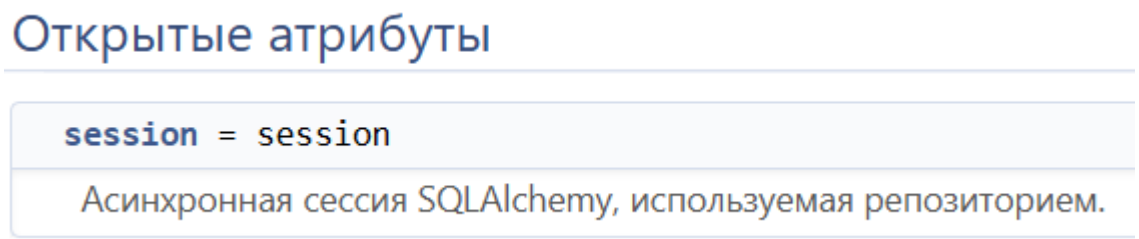


Рисунок 4 – Описание атрибута session класса TagRepository

Комментарий поясняет роль зависимости класса.

2.8 Описание проекта

Главная страница Beatstore Backend содержит название и краткую характеристику проекта: *Python backend: domains, API and infrastructure*. В подробном описании указан API музыкального каталога и используемые технологии: Python/FastAPI, SQLAlchemy Async/PostgreSQL, Pydantic и S3. Далее перечислены разделы архитектуры, работы с тегами, базы данных и общих компонентов (рисунок 5).

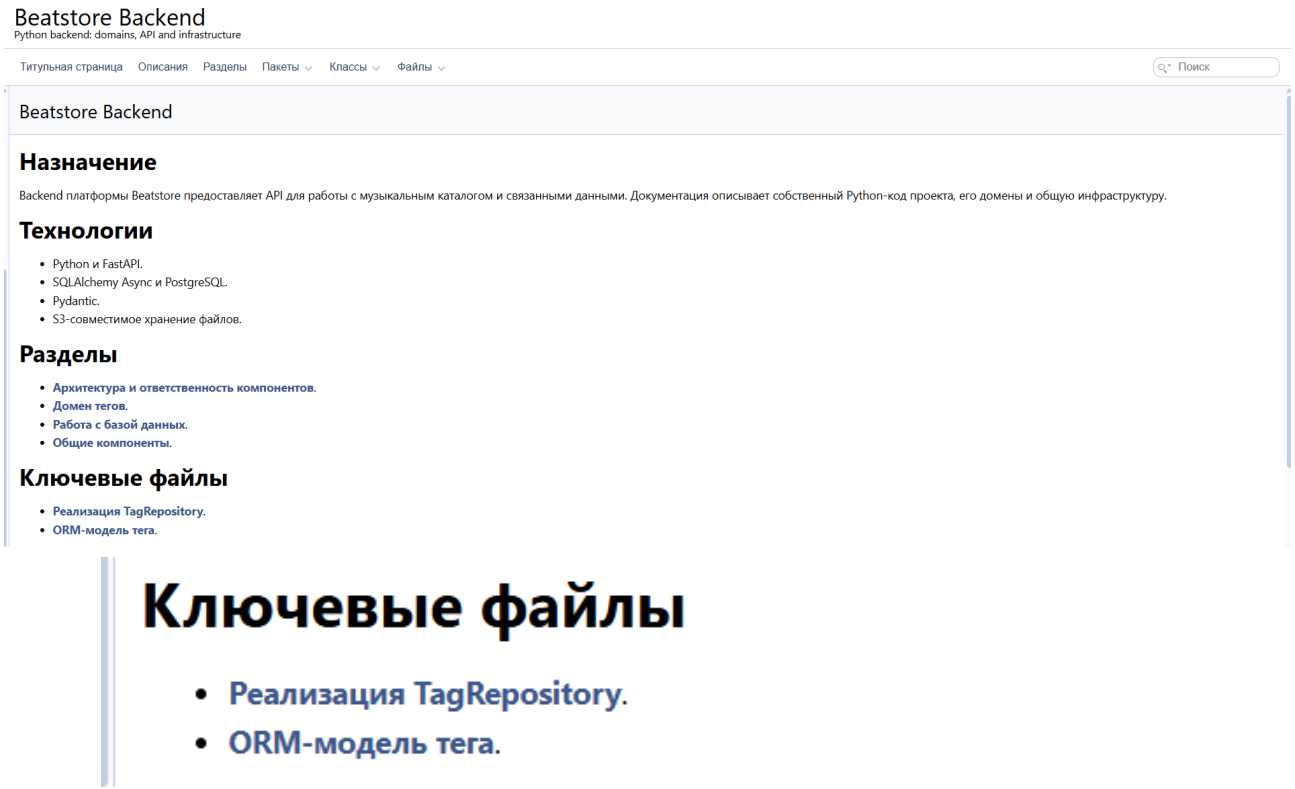


Рисунок 5 – Главная страница проекта и увеличенный фрагмент ссылок на файлы

Ссылки «Реализация TagRepository» и «ORM-модель тега» размещены в разделе «Ключевые файлы». Они связывают описание архитектуры с исходными файлами. Главная страница задаётся командой `@mainpage`, а ссылки на объекты и файлы — командой `@ref` [3].

2.9 Описание файла и ссылка на него в описании проекта

В начале `repository.py` расположен следующий блок:

@file
@brief Репозиторий для чтения тегов из базы данных.

На рисунке 6 показана страница файла. Она содержит его назначение, класс `TagRepository`, пространство имён `repository` и ссылку на исходный текст.

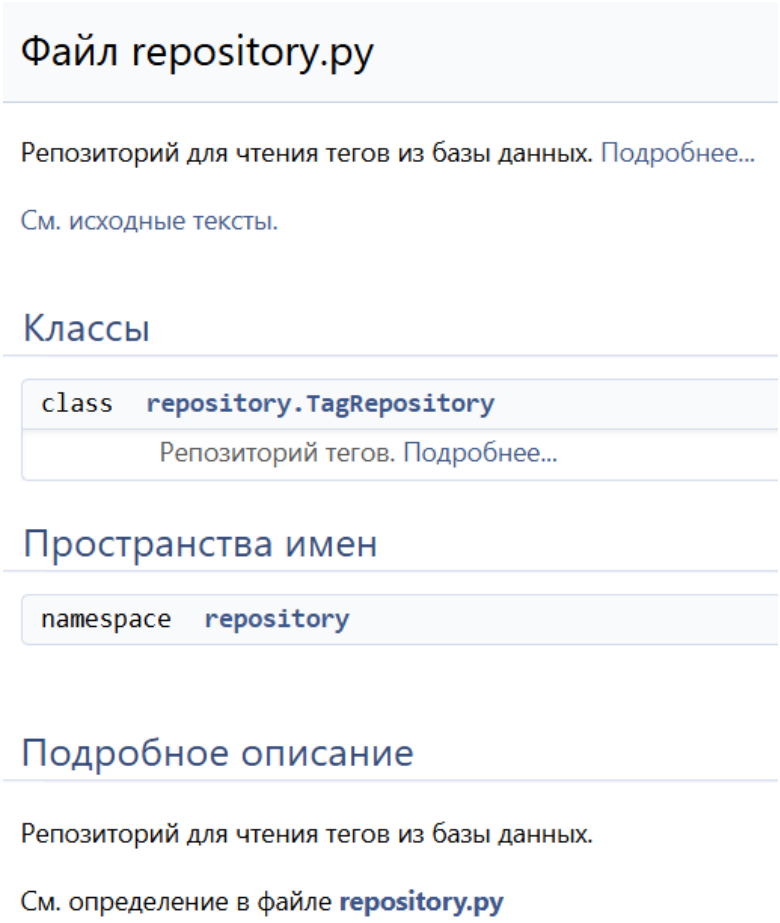


Рисунок 6 – Описание файла repository.py

Ссылка на реализацию репозитория на главной странице проекта (рисунок 5) обеспечивает переход от общего описания к файлу.

2.10 Описание производного класса

В качестве производного класса рассмотрена модель `Product`. На рисунке 7 видны её базовые классы `IdIntPKMixin`, `CreatedAtMixin` и `Base`, а также граф наследования.

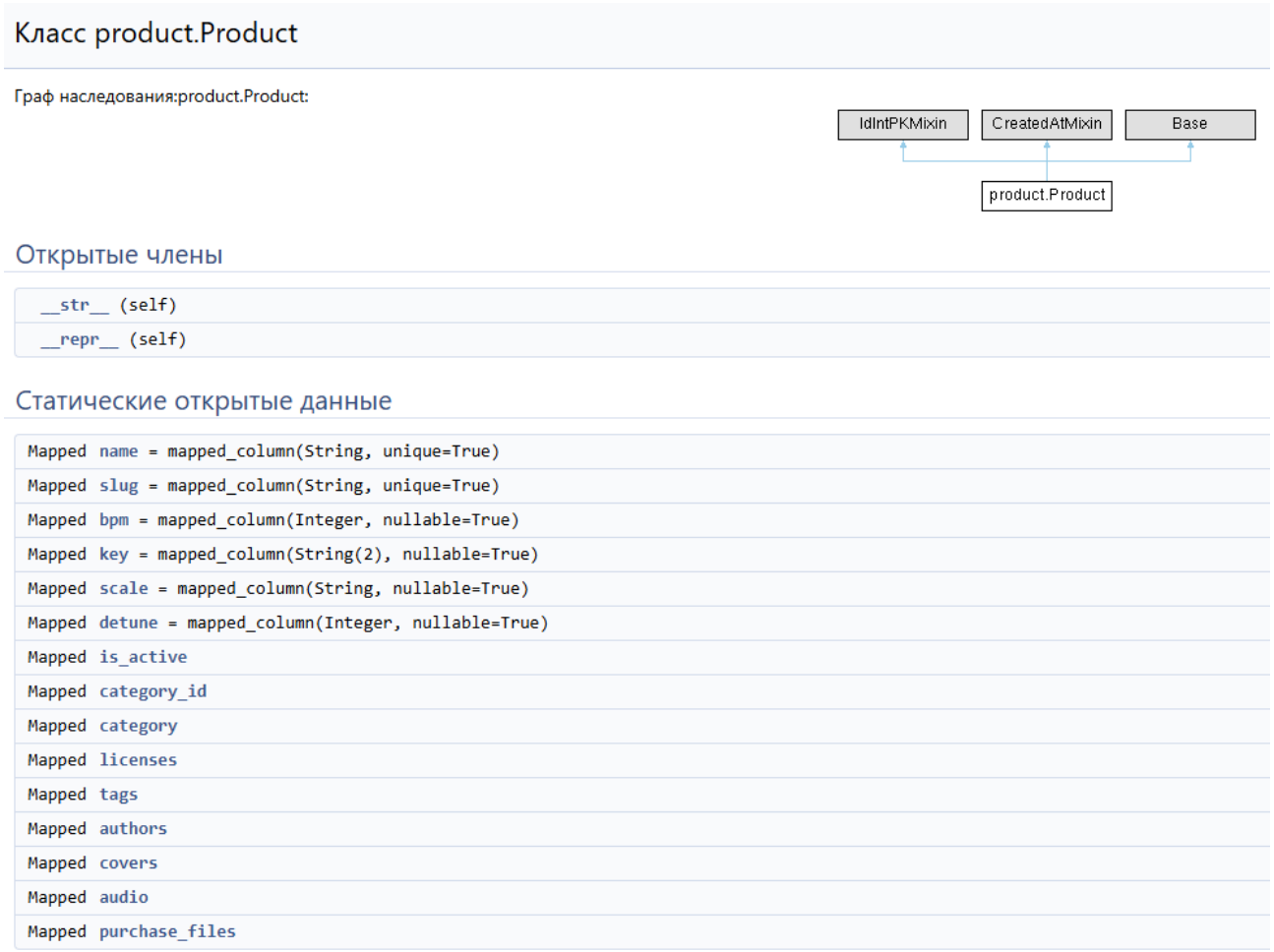


Рисунок 7 – Страница производного класса Product и граф наследования

Страница содержит методы `__str__` и `__repr__`, поля модели и связи с категориями, лицензиями, тегами, авторами и файлами каталога. Doxygen отображает состав производного класса и его место в иерархии.

3 Дополнительное задание

GitHub Copilot использован как интеллектуальный помощник при реализации методов классов в Visual Studio Code [5]. В дополнительном задании показано получение предложений кода по четырём видам запросов: краткому комментарию и имени функции, подробному описанию функции, описанию алгоритма и описанию вызова функции.

3.1 Подготовка проекта и класса

Для выполнения задания использован проект Beatstore Backend, написанный на Python. Работа выполнялась в Visual Studio Code с расширением GitHub Copilot. В редакторе подготовлен класс `TagRepository`, предназначенный для чтения тегов из базы данных. Конструктор принимает асинхронную сессию SQLAlchemy `AsyncSession` и сохраняет её в атрибуте `session`. Методы репозитория формируют запросы к ORM-модели `Tag` и выполняют их через эту сессию.

Для демонстрации выбраны операции получения списка тегов и поиска тега по имени и идентификатору. Дополнительно использованы методы сервисного слоя, которые обращаются к репозиторию через `self.db.tags`.

3.2 Краткий комментарий и имя функции

Первый запрос содержит краткое описание назначения метода и его сигнатуру:

```
# Получить список всех активных тэгов.
async def get_all_active_tags(self) -> list[Tag]:
```

На рисунке 8 Copilot формирует запрос `select(Tag)` с условием `Tag.is_active == True`. Затем запрос выполняется асинхронно через `self.session.scalars(tags_stmt)`, а полученные ORM-объекты преобразуются в список. Предложение согласуется с кратким комментарием: условие ограничивает выборку активными тегами, а возвращаемое значение соответствует аннотации `list[Tag]`.

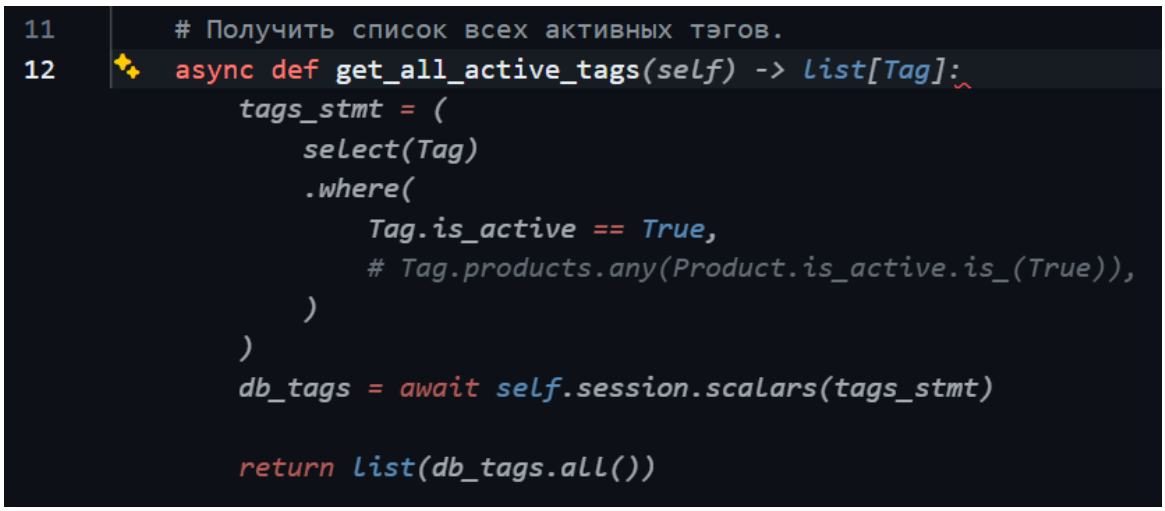


Рисунок 8 – Предложение Copilot по краткому комментарию и имени функции

3.3 Подробное описание работы функции

Для поиска тега по имени подготовлен более подробный комментарий, задающий условия выборки и ограничения реализации:

```
# Асинхронно найти тег, сравнив поле name с переданным name через SQL =.
# Использовать SQLAlchemy 2.x select и текущую AsyncSession.
# Вернуть ORM-объект Tag или None, если совпадений нет.
# Если найдено несколько строк, не скрывать нарушение
# ожидаемой уникальности.
# Не выполнять commit, не создавать новую сессию
# и не загружать products.
async def get_tag_by_name(self, name: str) -> Tag | None:
```

Метод принимает строку имени тега и возвращает объект `Tag` либо `None`.

На рисунке 9 показан код поиска по имени. Запрос включает выборку `select(Tag)` и условие `Tag.name == name`. Copilot использует текущую сессию и оператор `await`, не добавляет фиксацию транзакции или загрузку связанных продуктов. Однако получение результата через `first()` выполняет только часть условий запроса.

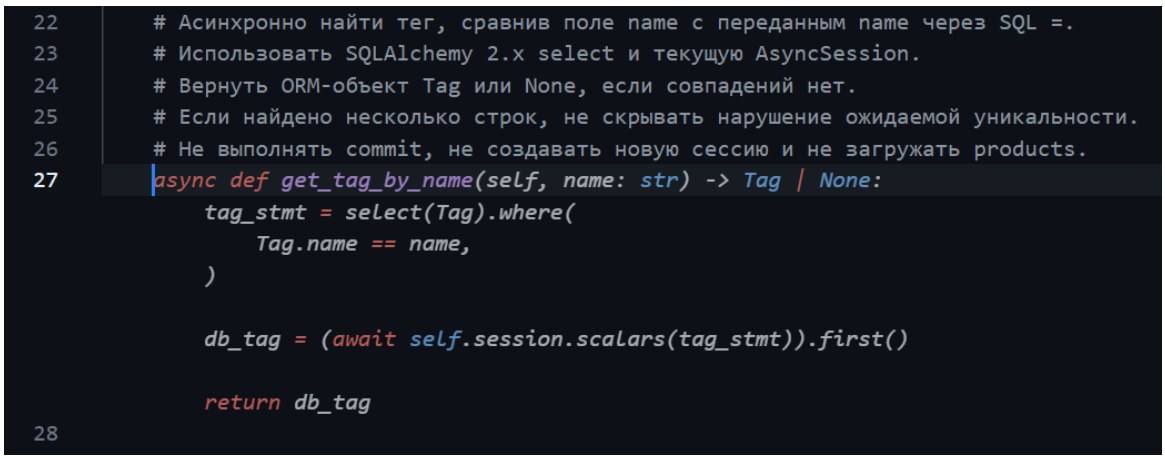


Рисунок 9 – Предложение Copilot по подробному описанию поиска тега

3.4 Комментарий с описанием алгоритма

Следующий вариант запроса описывает алгоритм получения тега по идентификатору. В данном проекте последовательность действий основной программы представлена методом сервисного слоя `get_tag_by_id`, который использует репозиторий. В комментарии заданы следующие шаги:

```
# Алгоритм получения тега по идентификатору:
# 1. Вызвать асинхронный метод get_tag_by_id(tag_id)
#    репозитория TagRepository.
# 2. Если репозиторий вернул None, вызвать TagNotFoundError
#    с сообщением ”Tag not found” и details={”tag_id”: tag_id}.
# 3. Иначе вернуть найденный объект тега.
```

Предложенный фрагмент на рисунке 10 повторяет эту последовательность: выполняет `await self.db.tags.get_tag_by_id(tag_id)`, проверяет результат оператором `if tag is None`, формирует заданное исключение и возвращает найденный тег. Алгоритм достаточно конкретен, чтобы помощник воспроизвёл порядок операций и данные ошибки.

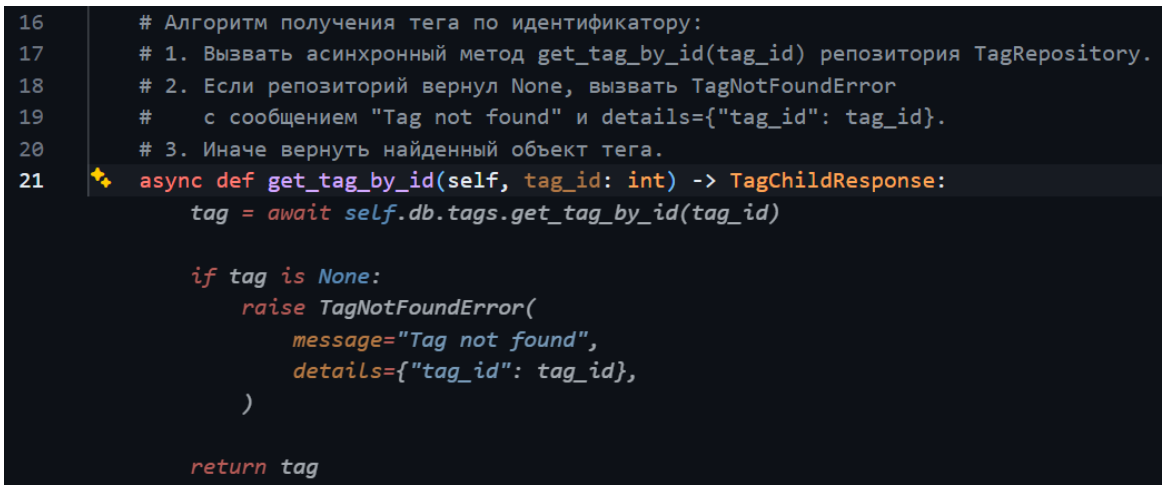


Рисунок 10 – Предложение Copilot по описанию алгоритма сервисного метода

В сигнатуре указан тип `TagChildResponse`. Тело метода возвращает переменную `tag`, полученную от репозитория. Этот фрагмент демонстрирует реализацию описанного алгоритма; соответствие фактического объекта аннотации результата необходимо учитывать при проверке метода в контексте проекта.

3.5 Комментарий с описанием вызова функции

Для последнего варианта запроса указан требуемый вызов метода репозитория из сервисного слоя:

```
# Получить список всех активных тэгов.
# Использовать вызов соответствующего метода из репозитория TagRepository.
```

На рисунке 11 приведён метод получения списка тегов. Его имя — `get_all_active_tags`, тип результата — `list`. В теле выполняется асинхронный вызов `self.db.tags.get_all_active_tags()` и возвращается список `tags`. Описание вызова позволяет задать связь между слоями проекта: сервис передаёт получение данных репозиторию, вместо повторного формирования SQL-запроса.

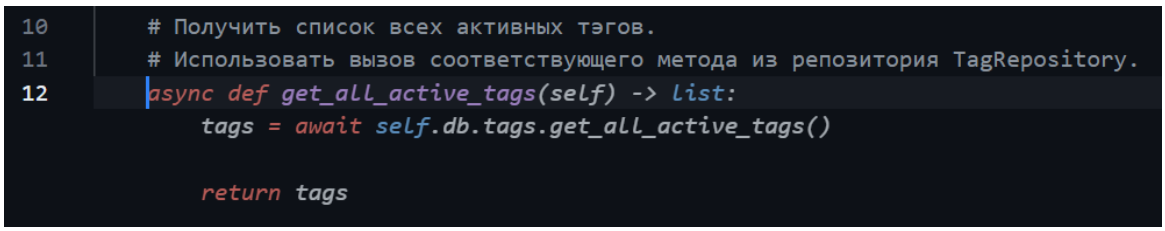


Рисунок 11 – Предложение Copilot по описанию вызова метода репозитория

3.6 Выбор и проверка предложенного кода

При сопоставлении предложений с запросами проверены состав SQL-запросов, использование текущей асинхронной сессии, наличие `await`, обработка отсутствующего тега и способ получения результата. Для каждого из четырёх вариантов запроса получен содержательный фрагмент реализации. Краткий комментарий задаёт назначение, подробное описание добавляет ограничения, алгоритм определяет порядок действий, а описание вызова указывает взаимодействие с репозиторием.

4 Заключение

В основной части лабораторной работы рассмотрено автоматическое документирование Python-проекта Beatstore Backend с помощью Doxygen. Подготовленные комментарии связывают исходный код со справочными страницами: краткие и подробные описания объясняют назначение классов и методов, описания параметров и результатов задают интерфейс функций, а описание атрибута `session` поясняет зависимость репозитория от сессии базы данных.

В отчёте представлены HTML-страницы класса `TagRepository`, его методов и атрибутов, проекта и файла `repository.py`, а также страница производного класса `Product` с графом наследования. Рассмотрены настройки генерации HTML и RTF.

В дополнительном задании продемонстрированы четыре способа задания контекста GitHub Copilot: краткий комментарий с именем функции, подробное описание работы функции, описание алгоритма и описание вызова другого метода. Помощник предлагает код с учётом используемых типов и асинхронных вызовов, однако результат требует проверки. В предложении поиска по имени применение `first()` расходится с требованием выявлять нарушение уникальности; для этого требуется получение единственного результата с проверкой числа совпадений.

Полученные навыки позволяют поддерживать справочную документацию рядом с кодом и использовать предложения интеллектуального помощника при разработке. Качество обоих результатов зависит от точности комментариев и проверки соответствия описания фактической реализации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Виноградова М. В., Королев С. В. Методические указания к лабораторным работам по курсу «Технологии разработки программ». Лабораторная работа №3 «Автоматическая генерация исходного кода программы и проектной документации» / под ред. М. В. Виноградовой. — М.: МГТУ им. Н. Э. Баумана, 2022.

2. Documenting the code // Doxygen Manual: [сайт]. — URL: https://www.doxygen.nl/manual/docblocks.html (дата обращения: 12.10.2026).

3. Special Commands // Doxygen Manual: [сайт]. — URL: https://www.doxygen.nl/manual/commands.html (дата обращения: 12.10.2026).

4. Configuration // Doxygen Manual: [сайт]. — URL: https://www.doxygen.nl/manual/config.html (дата обращения: 12.10.2026).

5. About GitHub Copilot // GitHub Docs: [сайт]. — URL: https://docs.github.com/en/copilot/get-started/about-github-copilot (дата обращения: 12.10.2026).

6. Working with Engines and Connections // SQLAlchemy 2.0 Documentation: [сайт]. — URL: https://docs.sqlalchemy.org/en/20/core/connections.html (дата обращения: 12.10.2026).